



DAY 3 · MODULE 13 · 90 MIN

Loop Engineering: 写控制程序，不是写提示词

Loop 的本质是传统控制程序——难点全在停止条件、熔断、状态持久化



我已经不直接 prompt Claude 了——我写 loop 让它去 prompt Claude。

AI 能写 loop 骨架，但停止条件 / 熔断 / 记忆这三个工程判断，它替你做不了。

我已经不直接 prompt Claude 了，我写 loop 让它去 prompt Claude

矩阵导航：你在「Loop」站——把 Day1-3 建的验证 / 状态 / gate，用一个 while 循环串起来自动跑

“我现在已经不直接 prompt Claude 了。我的工作变成写各种 loop，让这些 loop 去 prompt Claude，并弄清楚下一步该干什么。”

— Boris Cherny, Claude Code 创造者

Prompt/Context Engineering

保姆模式 · 人在循环内

- 你手动喂提示
- 看它生成的结果
- 报错了你复制回去再喂
- 你是 agent 的保姆，一步步带

Loop Engineering

loop 模式 · 人在循环外

- 你写一个自动循环系统
- 它自己喂提示、校验结果
- 失败就把错误当新提示回灌
- 你只在成功/卡住时介入

关键认知：Loop 的本质不是 prompting，是写一段传统的控制程序。

它就是一个带停止条件的 while 循环

```
context = 初始任务
for i in range(max_iterations):
    # 熔断: 最大轮数
    patch = call_agent(context) # agent 生成
    apply(patch)
    result = run_verification() # 裁判 (M12!)
    if result.success:
        create_pr(); return SUCCESS
    else:
        context = result.error_log
        # 失败信息=下一轮 prompt
        if cost > budget: return ABORT # 熔断:预算
return FAILED_CALL_HUMAN # 兜底:叫人
```

循环里的三个要素，全是我们学过的：

裁判 = M12 验证

run_verification() — 没有可靠裁判，loop 就是垃圾生产机

失败=下轮 prompt

error_log 回灌 — M12 的 test as prompt

熔断 + 叫人 = M10 gate

max 轮数 / 预算 / 人类兜底

human out of the loop: 人完全不在循环内——只在它成功提 PR、或熔断喊人时才介入。loop 不神秘，它复用了你这三天建的一切。

写 loop 的 90% 工夫，在这三件事上

停止条件

Stop Condition

什么是决定性的“完成”？

测试 100% 过 / 0 漏洞。错的停止条件 → 无限循环

熔断

Circuit Breaker

强制刹车，谁先到谁停

最大轮数 + token 预算 + 时间上限 → 防失控烧钱

外部记忆

State

loop 每轮失忆，靠 STATE.md

记“试过 A、B 都败，这轮试 C” → 防反复犯同样错

这三件事，没有一件是 prompting，全是工程判断。

而且全依赖你的业务上下文——AI 能写 loop 骨架，但你的停止条件、预算、什么算“试过”，它替你定不了。这就是 Loop Engineering 里 AI 给不了的核心。

连续 3 轮自信修复全失败，第 4 轮被迫“先画状态机”才对

任务听起来很简单：守护进程自升级——停进程 → 换二进制 → 重启。但 launchd 状态机有~12 个有意义的状态。

轮	修法	为什么挂（撞上没考虑的状态转移）
1 X	SIGTERM 优雅停止	SSE 长连接拖住进程，永远不退
2 X	改用 SIGKILL 强杀	launchd KeepAlive 立刻重启旧二进制 → 与 rsync 抢文件、I/O 损坏
3 X	bootout 注销服务	bootout 内部又发 SIGTERM（回到第1轮）→ 20s 超时、状态未定义
4 ✓	先画完整状态机再写代码	SIGKILL→bootout→rsync→bootstrap，每步依赖前一步状态，一次成功

根因：增量式“修症状而不理解系统”

每轮只建模 3-4 个状态，被第 5 个打脸。

“确定能 work”与正确性负相关：3 次都宣称确定，3 次全错。信心 ≠ 理解。

2-STRIKE 规则（直接进 loop 设计）

同一类修复连续失败 2 次 = 心智模型错了。停止加代码，先去理解系统 / 画状态机——别让 loop 第 3 次还在原地换花样。

没有验证的 LOOP = 全自动垃圾生产机

Loop 把验证的重要性放大了一万倍

手动模式

人在循环里

- 验证不太可靠也没关系
- 你用肉眼能兜底
- “这代码看着不对劲”

人眼是最后一道关

Loop 模式

人不在循环里

- 验证是唯一的对错判官
- 验证说通过，loop 就提 PR
- 被同义反复测试糊弄 → 自信地批量生产错误代码

错题本会被自动灌满“做对的题”

上 loop 之前，先问自己一个问题：

我的验证体系，敢不敢当一个无人值守的、唯一的判官？

不敢，就回去先把 M12 的功课补好。验证不到位，loop 就是个放大器——把你的错误放大一万倍。

别被吓退——从最轻量的本地 loop 起步

级别	形态	基建要求	适合
L1 本地 loop	笔记本上跑：契约断言红了 → agent 改到绿	几乎零额外基建	培养 TDD + loop 习惯
L2 CI loop	草稿 PR 触发，CI 里 agent 自修	GitHub Actions 等	把修琐碎 bug 外包给系统
L3 生产自愈 loop	线上识别报错 → 自动定位修复提 PR	监控+沙箱+可观测+webhook	系统级自动响应（M14 详谈）

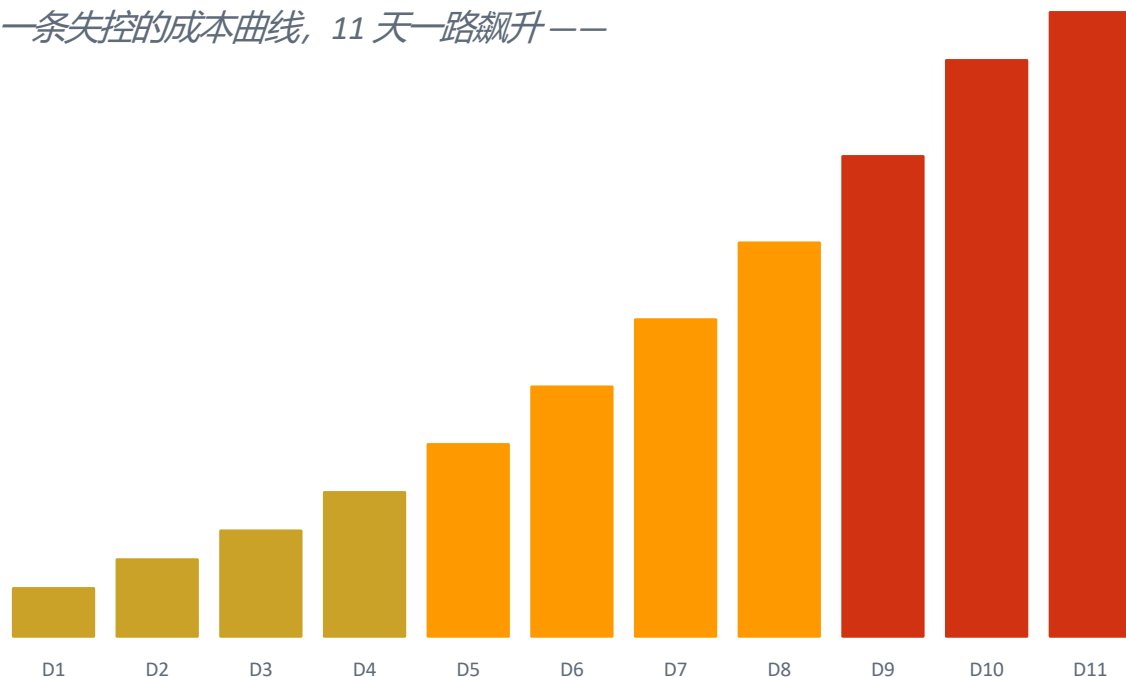
👉 今天 Lab 就做 L1：不需要改动公司大基建，在笔记本上让 agent 自动修一条识别契约断言。

务实建议：绝大多数团队从 L1 起步，跑通了、把验证练扎实了再往上爬。别一上来就想搞 L3，那是给自己挖坑。

真实事故 · 11 天烧几万美金

一个写错停止条件的 Loop，能烧掉你一个季度的预算

一条失控的成本曲线，11 天一路飙升——



💰 ~\$ 数万 API 费用，11 天无人发现

失败链：三个洞叠在一起 = 灾难

停止条件不可达 设成永远达不到的标准 → 永不自停

没有 token 熔断 烧钱没有上限

没人盯着 11 天没人喊停

产出：过度重构的“自信的废话”——语法完美、逻辑自洽，却完全偏离目标。

熔断不是可选项，是 loop 的安全带——它直接回应了管理层最痛的那个点：token 燃烧焦虑。

让 Loop 又快又省的几根杠杆

错题本 loop 的双重烧钱：① loop 调 Claude Code 改代码烧 token ② 验证里若真调 Bedrock 又烧一份。两头都要管。

验证金字塔省钱

每轮只跑确定的结构+契约断言 (mock, 秒级免费)，绝不每轮真调 Bedrock (M12/M7)

失败信息精炼

富而不冗的错误信息 = 更少轮数收敛 → 轮数少，钱就少

外部记忆避免重复试错

记住试过什么失败了，不再一遍遍烧钱犯同一个错

熔断三件套

max iterations / token budget / 时间上限 —— 安全带

模型路由 (伏笔 M14)

改小契约用小模型，难的识别逻辑重构才上大模型

loop 的工程水平，很大程度上就体现在它省不省钱上。
一个又对又省的 loop，才是生产级的 loop。

动手——在本地跑通你的第一个 Worker Loop (L1)

🏅 human out of the loop (但有刹车) —— 让 loop 自动修复错题本识别功能的一个真 bug

STEP 1

准备停止条件 + 一个 bug

用 M12 的 ci/verify.sh 当停止条件
+ 一条故意打破的识别契约断言

STEP 2

写最小 loop 控制脚本

调 Claude Code → 跑 verify → 不过则把错误回灌再来

STEP 3

加熔断 (先装刹车)

max iterations + 时间上限 + 叫人兜底 (错题本 loop 双重烧钱, 刹车尤其重要)

STEP 4

加外部记忆 STATE.md

记“试过什么失败”避免重复试错
; 跑通一次完整循环

安全第一：Step 2 熔断必须先做，再放手长跑（识别 loop 两头烧钱）。

复用 M12 的 verify.sh 当停止条件、M10 的 exit-contract 思想做 STATE.md。产出进 Capstone repo 的 loop/: worker-loop.sh + STATE.md。



HANDS-ON LAB · 45 MIN

Lab 13

跑通识别功能的第一个 Worker Loop (L1)

工具：Claude Code（唯一，由 loop 脚本调用） · 前置：harness repo 已有 ci/verify.sh（M12）、M9 识别实现
Claude Code 可被脚本调用（headless / print 模式） · 4 个 Step：准备 / 最小 loop / 熔断 / STATE.md
产出：loop/worker-loop.sh + loop/STATE.md，能在熔断保护下自动修复识别功能的 bug

Step 0-2: 准备停止条件 → 写最小 loop → 先装刹车

STEP 0-1 · 准备 bug + 写最小 loop 脚本

```
# Step0: 确认 verify 决定性, 再打破一条契约
./ci/verify.sh; echo "exit=$?" # 当前应通过
# 删坏 confidence<0.7 标待确认的逻辑 → 应失败

# loop/worker-loop.sh (Step1)
TASK="修复 src/ 识别 bug, 使 verify 契约全过;
      不要改测试、不要改断言。"
CONTEXT="$TASK"
for i in $(seq 1 5); do
  claude -p "$CONTEXT" \
    --dangerously-skip-permissions 2>/dev/null
  if OUT=$(./ci/verify.sh 2>&1); then
    echo "✅ 第 $i 轮收敛"; exit 0
  else CONTEXT="$TASK\n上轮失败:\n$OUT"; fi
done
```

STEP 2 · 加熔断三件套 (先装刹车)

```
MAX_ITER=5
MAX_SECONDS=300 # 时间上限 5 分钟
START=$(date +%s)
for i in $(seq 1 $MAX_ITER); do
  NOW=$(date +%s)
  if [ $(NOW-START) -gt $MAX_SECONDS ]; then
    echo "🕒 时间熔断, 叫人"; exit 2
  fi
  # ... 调用 + 验证 (同 Step1) ...
done
echo "🔄 轮数熔断, 叫人介入"; exit 1
```

退出码区分停止原因 (便于上层判断)

0 = 成功收敛 · 1 = 轮数熔断 · 2 = 时间熔断

宁紧勿松: 修不好时在熔断处主动停, 绝不无限跑、不失控烧钱。

核心原则: 先装刹车再放手——识别 loop 两头烧钱, 没有熔断就是 11 天事故的复刻。

Step 3: 加外部记忆 STATE.md, 跑通完整循环, 再复盘

STEP 3 · 10min 加外部记忆 STATE.md

每轮把“试了什么、为何失败”追加进 STATE.md, 下一轮读入 → agent 不再重复已失败的方向 (呼应 M10 exit-contract、Slide 5 的 2-strike)。

复盘 (3 min)

- ① 看着 loop 自己修、你不插手, 是什么感受? 敢让它修多复杂的 bug?
- ② 没先加熔断就放手——两头烧钱最坏烧成什么样?
- ③ 只验 mock、不验真识别质量, 修出来的敢直接合吗?
- ④ 升到 L2 (CI) / L3 (线上自愈) 各差什么? (M14 入口)
- ⑤ STATE.md 怎么帮 loop 在“2-strike”时停下换思路? 要不要加“连续 N 轮零进展就熔断叫人”?

```
# 每轮失败后追加进 STATE.md
echo "## 第 $i 轮 ($(date +%T))" >> loop/STATE.md
echo "尝试: <本轮修改方向>" >> loop/STATE.md
echo "结果: 失败 - $(echo "$OUT"|head -3)" \
>> loop/STATE.md

# 把历史纳入下一轮 CONTEXT
CONTEXT="$TASK
历史尝试 (不要重复这些已失败方向) :
$(cat loop/STATE.md)
本轮请基于以上教训修正。最近错误:
$OUT"
```

Done 标准

✓ 每轮尝试与失败记入 STATE.md ✓ 下一轮 prompt 含历史、不重复试错 ✓
STATE.md 可读、能复盘 ✓ loop/ 已提交进 harness repo